

Boids Parallelization

Matteo Tinacci

matteo.tinacci@edu.unifi.it

Abstract

*This project analyzes the performance optimization of a Boids simulation, whose brute-force flocking algorithm presents a computational bottleneck with a complexity of $O(N^2)$. The adopted approach was shared-memory parallelization using OpenMP, comparing different parallelization strategies (targeting Stage 1, Stage 2, or both) and various scheduling policies (static vs. dynamic). The key results, obtained on a **Quad-Core CPU with Hyper-Threading**, show a maximum speedup of $4.77\times$ with 8 threads. The analysis demonstrates that while Hyper-Threading provides a tangible performance boost beyond the physical core count, software oversubscription (16 threads) becomes counterproductive due to context-switching overhead. Furthermore, the data reveal that minimizing synchronization overhead by merging parallel regions proved to be the most effective optimization strategy, confirming static scheduling as the optimal choice for this homogeneous workload.*

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

The *Boids* simulation [1] is used to realistically model the collective motion of flocks of birds, fish schools, or crowds. The overall complex behavior emerges from the application of three simple rules that each agent (or “boid”) follows in a decentralized manner:

1. **Separation:** Avoid crowding with neighbors in the area.
2. **Alignment:** Match the direction of motion with the average heading of nearby agents.
3. **Cohesion:** Move toward the average position of nearby agents.

1.1. The Computational Problem

The most straightforward implementation—the *brute-force* approach—presents a significant performance issue. To correctly apply the three rules, each boid must compute its distance and relative velocity with respect to each other boid in the simulation.

This approach introduces a computational bottleneck with a complexity of $O(N^2)$, where N is the number of boids.

Such quadratic growth makes brute-force simulation unsustainable for a large number of agents, limiting its usefulness in complex scenarios. However, this *embarrassingly parallel* nature—where the computation of each boid is (initially) independent of the others—makes it an ideal candidate for optimization through parallel computing.

1.2. Objective

The goal of this project is to optimize the $O(N^2)$ simulation by exploiting the shared-memory parallelism provided by OpenMP. Various strategies for parallelizing the workload across a multi-core CPU are analyzed, measuring their impact on performance and scalability. The analysis focuses not only on reducing execution time (speedup), but also on the limits imposed by Amdahl’s Law, thread contention (Hyper-Threading) and advanced hardware factors such as cache locality and synchronization overhead.

2. Methodologies and Test Environment

To ensure the reproducibility of the results, we report here all the resources and settings used for

the experiments.

2.1. Hardware & Software

All benchmarks were executed on the same machine to ensure data consistency. The specifications of the test environment are as follows.

1. **Hardware:** HP Pavilion Power Laptop 15-cb0xx
2. **CPU:** Intel Core i7-7700HQ @ 2.80GHz
 - Architecture: x86_64 (Kaby Lake)
 - Physical Cores: 4
 - Logical Threads: 8 (Hyper-Threading enabled)
 - L3 Cache: 6 MiB
3. **Operating System:** Ubuntu 22.04 LTS
4. **Compiler:** g++ (GCC) with OpenMP support
5. **Languages:** C++ (simulation), Python (data analysis)
6. **Libraries:** OpenMP, SFML (visualization only), Pandas, Matplotlib

2.2. Benchmark Methodologies

The primary objective of the test is to measure *strong scaling*: how performance changes as the number of processors (threads) increases, while keeping the problem size fixed.

1. **Fixed Problem Size:** All tests were executed with the same workload:
 - (a) **Number of Boids:** `NUM_BOIDS = 5000`
 - (b) **Number of Iterations:** `NUM_STEPS = 1000`
2. **Time Measurement:** The OpenMP high-resolution timer `omp_get_wtime()` was used. Timers were strategically placed to measure only the computation loop, excluding all I/O costs (console output) and initialization overhead (flock creation and memory allocation).

3. **Test Automation:** Scalability tests were automated using a custom **Python script**. To ensure statistical robustness, the script executes the `BenchmarkMain` binary **5 times** for each configuration (setting `OMP_NUM_THREADS` to 1, 2, 4, 8, and 16) and reports the arithmetic mean of the execution times. A *warm-up* phase was also implemented to mitigate cold-start effects.

4. **Data Analysis:** Results were saved as a `.csv` file and subsequently analyzed with a Python script using the Pandas library (for calculating speedup and efficiency) and Matplotlib (for generating plots).

2.3. Statistical Validity

Despite the significant execution time required for the simulations, a rigorous multi-run methodology was adopted to ensure data reliability:

1. **Multiple Iterations:** Each benchmark configuration was executed **5 times**. The values reported in this document represent the arithmetic mean of these runs, minimizing the impact of Operating System background noise.
2. **Warm-up Phase:** A preliminary warm-up run with a reduced workload ($N = 500$) was performed before the measurement cycle to load the necessary libraries into RAM and populate the CPU cache.
3. **Controlled Environment:** Background processes were minimized, and the laptop was connected to power to prevent frequency scaling issues.

3. Algorithm Description

The implementation of the Boids simulation in this project is based on a discrete update loop executed for a fixed number of `NUM_STEPS`. The core characteristic of the algorithm is a clear separation between the computation of behavioral rules and the application of movement. The algorithm was built using Hunter Adams'[2] one as a reference.

3.1. Algorithm Structure

Within each simulation step, the computation is strictly divided into two sequential phases (or “stages”):

1. **STAGE 1 – Rule Computation** (*calculateRules*): This is the computationally intensive phase. For each individual boid in the flock, the program iterates over all other boids to apply the three rules (Separation, Alignment, Cohesion). The result of this process is an acceleration vector (`boid.acceleration`) that determines the desired movement direction for that boid.
2. **STAGE 2 – State Update** (*updateState*): This is a lightweight phase. After all boids have computed their desired acceleration, a second loop iterates through the entire flock. In this phase, each boid applies its computed acceleration to update its velocity and, subsequently, its position (`boid.velocity` and `boid.position`).

3.2. Bottleneck Analysis

The two-phase division is essential for maintaining simulation correctness and preventing **data hazards**. If one boid (Boid A) were to compute its acceleration and immediately update its position ($S1+S2$), another boid (Boid B), still in the middle of its computation ($S1$), would “read” the new position of Boid A. This *dirty read* would break the integrity of the simulation step, as computations would no longer be performed on a consistent global state.

Therefore, two separate loops are employed:

1. The first loop (STAGE 1) represents the computational bottleneck. With N boids, each boid performs N checks, leading to an overall complexity of $O(N^2)$. This portion of the code accounts for the vast majority of the execution time.
2. The second loop (STAGE 2) is a simple $O(N)$ pass. It is fast, but it cannot begin until STAGE 1 has been completed.

4. Parallelization Strategies

Based on the two-phase structure (Stage 1 and Stage 2) of the algorithm, five different execution strategies were implemented and compared to evaluate the impact of OpenMP parallelization.

4.1. Sequential (Baseline)

The first version is the pure *brute-force* implementation. Both loops (Stage 1 and Stage 2) are executed on a single thread. This code contains no OpenMP directives and serves as the baseline for computing the speedup of all other strategies.

4.2. Parallel (Stage 1 Only)

This strategy focuses on optimizing the bottleneck identified in Stage 1.

1. **Stage 1:** Parallelized using the `#pragma omp parallel for` directive. The rule computation workload is distributed among all available threads.
2. **Stage 2:** Remains sequential.
3. **Objective:** This test directly demonstrates Amdahl’s Law. A significant speedup is expected, but it will be limited by the fraction of time still spent in the sequential Stage 2.

4.3. Parallel (Stage 2 Only)

This is a “control” strategy implemented to validate the bottleneck analysis.

1. **Stage 1:** Remains sequential.
2. **Stage 2:** Parallelized using `#pragma omp parallel for`.
3. **Objective:** The hypothesis is that this strategy will yield no benefit (or even degrade performance). Leaving the $O(N^2)$ bottleneck untouched while adding OpenMP overhead to the already lightweight $O(N)$ portion should prove ineffective.

4.4. Parallel (Stage 1 + Stage 2, Separate Regions)

This strategy parallelizes both computational stages using two separate consecutive *parallel for* directives.

1. **Stage 1:** Parallelized with `#pragma omp parallel for`.
2. **Stage 2:** Parallelized with a second `#pragma omp parallel for`.
3. **Objective:** In theory, this strategy incurs the *fork/join* overhead twice—once for each loop. It was tested to evaluate the impact of cache locality under this configuration.

4.5. Parallel (Stage 1 + Stage 2, Single Region)

This is the optimized implementation of Strategy 4, specifically designed to minimize OpenMP runtime overhead.

1. **Stage 1 & Stage 2:** A single `#pragma omp parallel` directive creates the thread team only once. Inside this persistent parallel region, two `#pragma omp for` directives (without the *parallel* keyword) distribute the workload for Stage 1 and Stage 2 among the already active threads.
2. **Objective:** The primary goal is to exploit **thread persistence**. Unlike Strategy 4, which incurs a "fork/join" synchronization cost for every loop, this strategy aims to eliminate that redundancy. The comparison with Strategy 4 serves to isolate and quantify the exact performance cost of OpenMP thread management versus the computational work itself.

5. Discussion of Results

The first analysis focuses on how execution times vary with the number of threads. The results are shown in Table 3 and in Figure 1.

Observing the Sequential, S1, and S2 strategies reveals the following:

1. The **Parallel (Stage 2 Only)** strategy produced execution times nearly identical to the

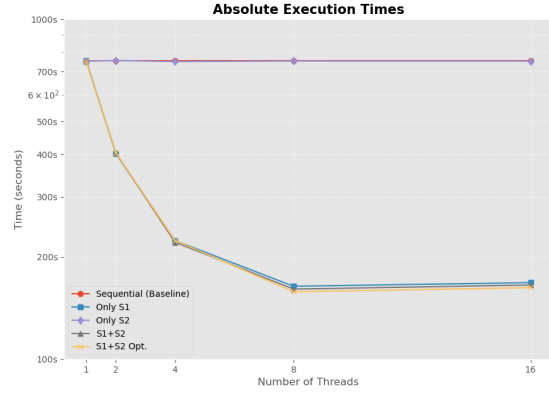


Figure 1. Execution time as a function of the number of threads.

sequential baseline (752.3 s vs 754.1 s), resulting in a flat Speedup of $1.00\times$. This confirms that Stage 2 (update of positions, $O(N)$) represents a negligible fraction of the total computational load compared to Stage 1 (interactions, $O(N^2)$). Parallelizing only this stage yields no tangible benefit.

2. The **Parallel (Stage 1 Only)** strategy drastically reduced the total runtime to approximately 164.0 s (with 8 threads). This confirms that the bottleneck was correctly identified. The discrepancy between the best S1-only time (164.0 s) and the best S1S2-Opt time (158.0 s) is approximately 6 seconds, representing the residual time taken by Stage 2 and other overheads.

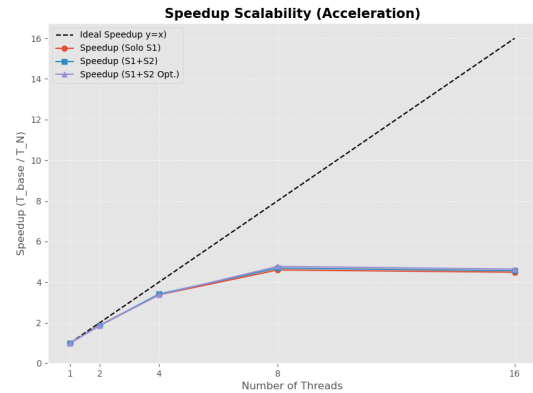


Figure 2. Speedup scalability as the number of threads increases.

Threads	Speedup S1	Speedup S2	Speedup S1S2	Speedup S1S2 Opt
1	1.00×	1.00×	1.00×	1.00×
2	1.87×	1.00×	1.86×	1.86×
4	3.37×	1.01×	3.41×	3.37×
8	4.60×	1.00×	4.69×	4.77×
16	4.49×	1.00×	4.56×	4.64×

Table 1. Speedup results (calculated against averaged sequential time).

Threads	Efficiency S1	Efficiency S2	Efficiency S1S2	Efficiency S1S2 Opt
1	1.00%	1.00%	1.00%	1.00%
2	0.94%	0.50%	0.93%	0.93%
4	0.84%	0.25%	0.85%	0.84%
8	0.57%	0.13%	0.59%	0.60%
16	0.28%	0.06%	0.29%	0.29%

Table 2. Efficiency $E(N)$ for the different configurations.

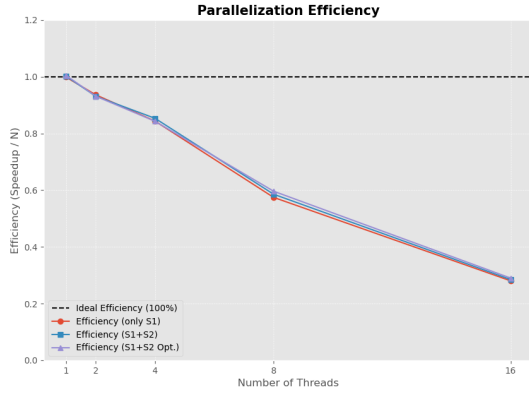


Figure 3. Efficiency as the number of threads increases.

By analyzing the speedup and efficiency results shown in Figures 2 and 3, and in Tables 1 and 2, we can correlate performance with the hardware architecture:

- Physical Cores Scaling (1–4 Threads):** The system exhibits excellent strong scaling up to 4 threads. At 4 threads, the speedup reaches **3.37×** with an efficiency of **84%**. This indicates that the workload is effectively distributed across the 4 physical cores with minimal overhead.
- Hyper-Threading Benefit (4–8 Threads):** Moving from 4 to 8 threads, the speedup continues to improve, reaching its peak at **4.77×**. However, the gain is not linear. This behavior is expected: the processor is utilizing

Hyper-Threading to execute two threads on each physical core. While this hides memory latency and improves instruction throughput, the physical execution units are shared, preventing a doubling of performance. The efficiency consequently drops to **60%**.

- 3. Oversubscription (8 vs 16 Threads):** Increasing the thread count to 16 results in a performance degradation (Speedup drops to **4.64×**). Since the hardware only supports 8 simultaneous logical threads, spawning 16 threads forces the Operating System to perform aggressive time-slicing and context switching. The overhead of managing twice as many threads as there are logical processors outweighs any potential load-balancing benefits.

A critical architectural insight emerges from the comparison between the standard **S1+S2** strategy (Strategy 4) and its **Optimized** single-region counterpart (Strategy 5).

The theoretical premise was that Strategy 5 would outperform Strategy 4 by minimizing the OpenMP runtime overhead. By using a single `#pragma omp parallel` region, the optimized version incurs the cost of thread team creation (`fork/join`) only once per simulation step, whereas Strategy 4 incurs it twice.

The experimental data **validates this hypothesis**: the optimized version (158.01 s) proved to

Threads	Time Seq (s)	Time S1 (s)	Time S2 (s)	Time S1S2 (s)	Time S1S2 Opt (s)
1	754.078	755.186	752.299	752.530	750.448
2	754.078	402.662	755.370	404.877	404.783
4	754.078	223.451	749.954	220.879	223.598
8	754.078	164.002	752.722	160.875	158.014
16	754.078	168.059	752.152	165.393	162.460

Table 3. Average execution times (in seconds) over 5 runs.

be superior to the multi-region version (160.88 s). Merging the loops successfully mitigated the synchronization overhead, yielding a net performance gain of approximately 2.8 seconds over the entire simulation.

Finally, a comparative analysis was conducted between **static** and **dynamic** scheduling policies. The underlying hypothesis was that the $O(N^2)$ brute-force algorithm generates a **perfectly homogeneous workload**: since every boid interacts with exactly $N - 1$ neighbors, the computational cost per iteration is identical for all agents.

To ensure a rigorous comparison, a dedicated test was performed applying *schedule(dynamic)* to the **Optimized (Single Region)** strategy with 8 threads. Experimental results confirmed the hypothesis: the dynamic configuration recorded an average runtime of **164.28 s**, which is slower than the equivalent static configuration (**158.01 s**). The performance gap (approx. 6.3 s) is attributable to the runtime overhead of dynamic load balancing (managing work queues), which provides no benefit in a scenario with zero load imbalance.

6. Conclusions

The Boids simulation was successfully parallelized using OpenMP, reducing the execution time from 754.08 s (Sequential) to **158.01 s** (Optimized Parallel).

The **Optimized S1+S2** strategy (using a *single parallel region*) achieved the best performance, reaching a maximum speedup of $4.77\times$ on 8 logical threads. This result is consistent with the hardware architecture of the test machine (Intel Core i7-7700HQ), which features **4 physical cores** with Hyper-Threading. The speedup exceeds the theoretical limit of physical cores ($4\times$) thanks to Hyper-Threading, although the gain is

not linear due to the contention for shared execution units. Conversely, pushing the parallelism to 16 threads proved detrimental due to *oversubscription*, confirming that exceeding the number of logical processors degrades performance in compute-bound tasks.

From a software design perspective, the experimental results highlighted two key findings:

1. **Synchronization Overhead:** Merging the two simulation stages into a single parallel region (Strategy 5) outperformed the multi-region approach (Strategy 4). This demonstrates that, for this specific workload, minimizing the *fork/join* overhead of threads is more critical than maximizing theoretical modularity.
2. **Workload Distribution:** The analysis confirmed that **static scheduling** is the superior policy for the $O(N^2)$ Boids algorithm. Since the workload is perfectly homogeneous, the runtime overhead introduced by dynamic scheduling provided no benefit and resulted in slightly higher execution times.

References

- [1] Craig W. Reynolds. Flocks, herds, and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '87, pages 25–34, New York, NY, USA, 1987. ACM.
- [2] V. Hunter Adams. Boids algorithm. Personal Web Site.